المرحلة الثانية:
كشف متعدد الفئات، الإخفاء الجزئي، ومقارنة خوارزميات الكشف

شرح كامل لما طلبه الأستاذ، مع الكود والتعليقات والنتائج

5	0.651	3100	9
فئات في ملف .pt واحد	mAP@0.5 على الاختبار	صورة (COCO + مسيّرات)	خوارزميات في المقارنة

ما طلبه الأستاذ

1. النموذج لا يكتشف الأشخاص فقط: أضف فئات أخرى (طائرات مسيّرة UAV، سيارات، قطط، كلاب) ووحّدها في ملف `.pt` واحد.
2. جزء من الشخص مخفي خلف جسم آخر: اكتشف **الجزء الظاهر** (Occluded object detection).
3. تعرّف على خوارزميات كشف أخرى غير YOLO.

المحتويات

1. بيانات متعددة الفئات
2. التسميات الزائفة (Pseudo-labeling)
3. تدريب النموذج الموحّد ونتائجه
4. الإخفاء الجزئي: الفكرة والكود
5. تجربة الإخفاء ونتائجها
6. خوارزميات الكشف الأخرى: النظرية
7. مقارنة الخوارزميات: النتائج
8. المشاكل التي واجهتنا
9. التشغيل بكاميرا الهاتف
10. أسئلة للمراجعة مع الإجابات

1. بيانات متعددة الفئات

الفئات الخمس ومصادرها

الرقم	الفئة	المصدر	صور التدريب	مربعات التدريب
0	person (شخص)	COCO 2017	2400 صورة (~600 لكل فئة)	3596
1	car (سيارة)	COCO 2017		1983
2	cat (قطة)	COCO 2017		600
3	dog (كلب)	COCO 2017		641
4	drone (مسيّرة)	Seraphim Drone Dataset (Hugging Face)، رخصة CC-BY-4.0	700 صورة	585

المجموع **3100 صورة**، مقسمة **2480** للتدريب، و**310** للتحقق، و**310** للاختبار. كل فئة موجودة في الأقسام الثلاثة.

> **لماذا مصدر آخر للمسيّرات؟**
> مجموعة COCO فيها 80 فئة، لكن ليس فيها فئة "drone". لذلك أخذنا صور المسيّرات من مجموعة بيانات عامة أخرى جاهزة بصيغة YOLO، وغيّرنا رقم فئتها من 0 إلى 4 حتى تأتي بعد فئات COCO.

أ) اختيار صور COCO لعدة فئات

`scripts/download_coco_multi.py`

```
def select_images(coco, class_names, per_class, min_area, seed=0):
    name_to_coco = {c["name"]: c["id"] for c in coco["categories"]}   # "cat" ← 17
    coco_to_idx = {name_to_coco[n]: i for i, n in enumerate(class_names)}  # 17 ← 2 رقمنا
    ...
    for ann in coco["annotations"]:
        idx = coco_to_idx.get(ann["category_id"])
        if idx is None:
            continue                                    # فئة لا نريدها ← تجاهلها
        if ann.get("iscrowd"):
            crowd.add(ann["image_id"])                  # صورة حشد ← سنستبعدها
            continue
        boxes.setdefault(ann["image_id"], []).append((idx, ann["bbox"]))   # نحفظ كل مربع، حتى الصغير
        if w * h >= min_area * img_w * img_h:
            big_enough[idx].add(ann["image_id"])       # الصورة مرشحة لهذه الفئة

    # صور الفئات النادرة (person) الفئة الأندر أولاً، حتى لا تأخذ الفئة الشائعة
    for idx in sorted(big_enough, key=lambda i: len(big_enough[i])):
        candidates = sorted(big_enough[idx] - crowd - chosen)
        rng.shuffle(candidates)
        chosen.update(candidates[:per_class])           # 600 صورة لكل فئة
    return {i: boxes[i] for i in chosen}, images
```

تصحيح خطأ من المرحلة الأولى

في النموذج الأول كنا نحذف مربعات الأشخاص الصغار، لكن نُبقي صورهم. النتيجة: أشخاص بلا علامة، فيتعلم النموذج أنهم "خلفية". الآن: الصورة تُختار إذا كان فيها جسم كبير بما يكفي (≤ 1% من الصورة)، لكن **كل** الأجسام من فئاتنا تحصل على علامة، مهما كان حجمها.

ب) إضافة مجموعة بيانات أخرى كفئة جديدة

`scripts/add_yolo_dataset.py`

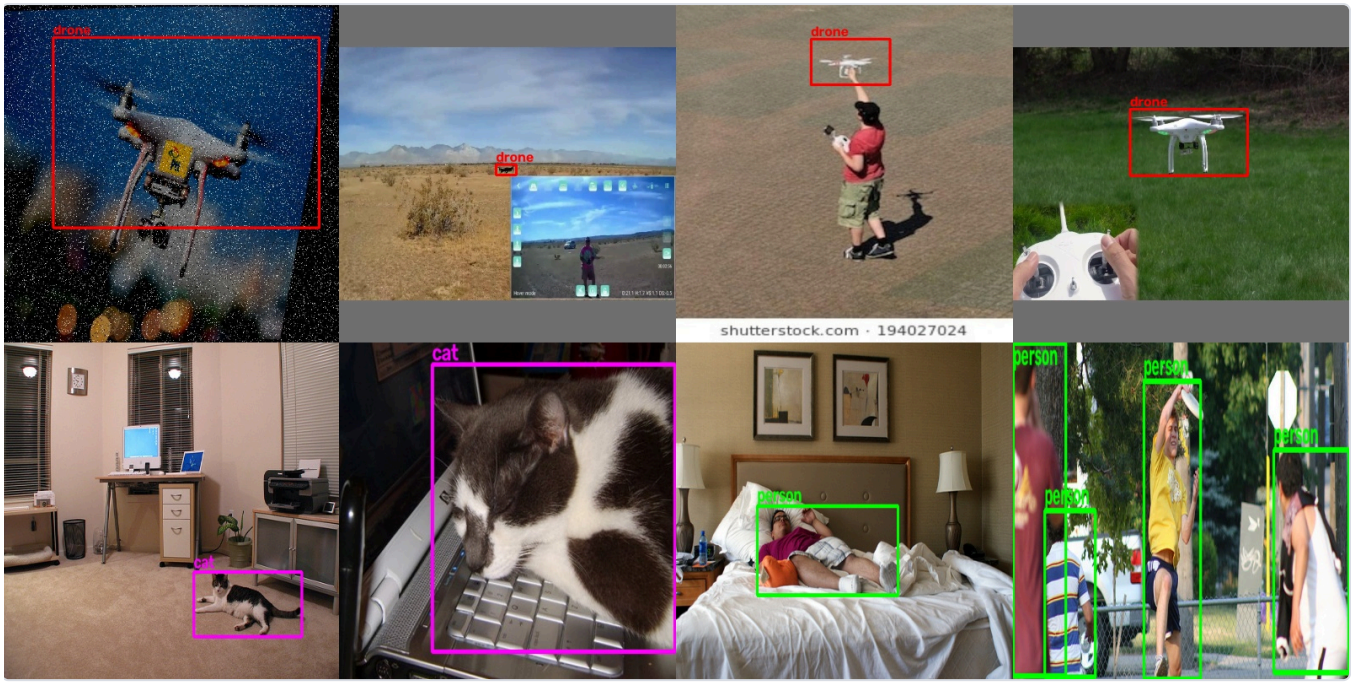
```
def read_label(path, class_id):
    lines = []
    for raw in open(path):
        parts = raw.split()
        if len(parts) != 5:
            return None                                 # سطر غير صالح ← نرفض الملف كله
        xc, yc, w, h = map(float, parts[1:])
        if not all(0.0 <= v <= 1.0 for v in (xc, yc, w, h)) or w == 0 or h == 0:
            return None                                 # أرقام خارج الصورة ← نرفض
        lines.append("%d %.6f %.6f %.6f %.6f" % (class_id, xc, yc, w, h))   # 0 ← 4
    return lines
```

نتحقق من كل ملف علامات قبل استعماله، لأن البيانات من مصدر خارجي. نستبدل رقم الفئة (0 في المصدر) بالرقم 4، وننسخ 700 صورة عشوائية مع علاماتها.

2. التسميات الزائفة (Pseudo-labeling)

المشكلة

مجموعة المسيّرات تضع علامة على المسيّرات فقط. لكن في بعض صورها أشخاص يمسكون جهاز التحكم، أو سيارات في الخلفية، **بدون علامة**. لو تدرّب النموذج عليها كما هي، سيتعلم أن "هذا الشخص خلفية"، وهذا يتعارض مع صور COCO.



عيّنة من البيانات مع العلامات. في الصورة الثالثة من الأعلى: شخص يمسك المسيّرة ولا علامة عليه في البيانات الأصلية.

الحل

نستعمل نموذج **YOLOv5s** المدرَّب مسبقاً على 80 فئة من COCO (يعرف الأشخاص والسيارات والقطط والكلاب)، ونمرّره على صور المسيّرات. كل كشف واثق (ثقة ≥ 0.5) من فئاتنا يُضاف كعلامة.

`scripts/pseudo_label.py`

```
COCO80_TO_OURS = {0: 0, 2: 1, 15: 2, 16: 3}   # رقم الفئة في COCO-80 ← رقمنا (person, car, cat, dog)

model = torch.hub.load(args.yolov5_dir, "custom", path="yolov5s.pt", source="local")
model.conf = 0.5                                  # نقبل فقط الكشوف الواثقة

for path in glob.glob("data/multi_raw/images/drone_*"):
    rgb = cv2.cvtColor(cv2.imread(path), cv2.COLOR_BGR2RGB)
    det = model(rgb, size=640).xywhn[0].tolist()   # مربعات بأرقام نسبية (0..1) جاهزة لصيغة YOLO
    new = ["%d %.6f %.6f %.6f %.6f" % (COCO80_TO_OURS[int(c)], x, y, w, h)
           for x, y, w, h, _, c in det if int(c) in COCO80_TO_OURS]   # فئاتنا فقط
    with open(label_path, "a") as f:              # "a" = لا نضيف إلى العلامات الموجودة، نمسحها
        f.write("\n".join(new) + "\n")
```

النتيجة

أُضيفت **188** علامة (أشخاص، سيارات...) إلى **137** صورة من صور المسيّرات.

ما هو Pseudo-labeling؟

استعمال نموذج قوي جاهز لوضع علامات على بيانات ناقصة، بدل وضعها باليد. هذه تقنية حقيقية تُستعمل في الأبحاث والصناعة. خطرها أن أخطاء النموذج المساعد تدخل البيانات، لذلك نقبل فقط الكشوف ذات الثقة العالية.

3. تدريب النموذج الموحّد ونتائجه

```
python scripts/split_dataset.py --src data/multi_raw --dst data/multi_dataset \
        --names person car cat dog drone           # 5 فئات بهذا الترتيب

python train.py --img 416 --batch 32 --epochs 30 \
        --data data/multi_dataset/data.yaml --weights yolov5n.pt --seed 0
```

نبدأ مرة أخرى من `yolov5n.pt` (التعلم بالنقل). رأس الكشف يتغير من 80 فئة إلى **5 فئات**. الناتج: ملف واحد `models/multi_yolov5n.pt` (3.8 MB) يكتشف الفئات الخمس معاً.

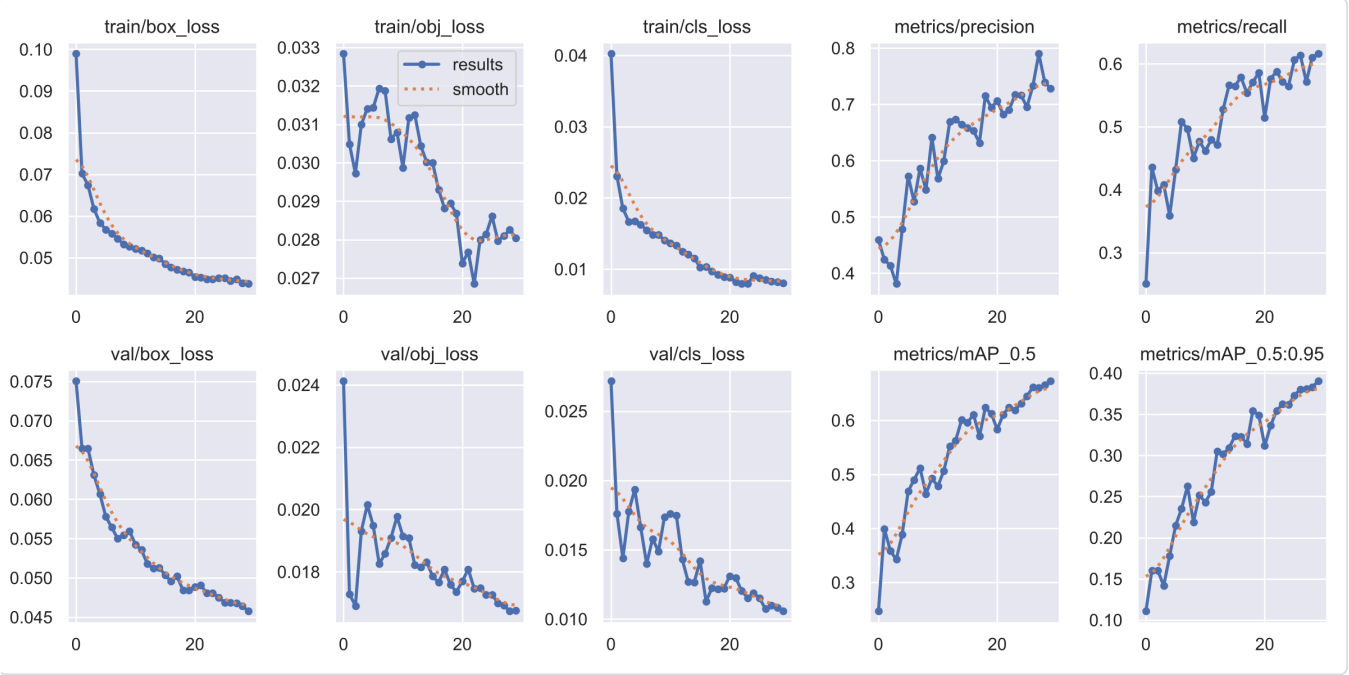
> **انقطاع أثناء التدريب، واستئنافه**
> أُعيد تشغيل الجهاز بعد الجولة 12 من 30. YOLOv5 تحفظ `last.pt` في نهاية كل جولة، فاستأنفنا من حيث توقفنا:
> `python train.py --resume runs/train/multi/weights/last.pt`

النتائج على صور الاختبار (310 صورة)

الفئة	Precision	Recall	mAP@0.5	mAP@0.5:0.95
شخص (person)	0.749	0.562	0.629	0.345
سيارة (car)	0.660	0.462	0.503	0.279
قطة (cat)	0.798	0.688	0.768	0.508
كلب (dog)	0.686	0.527	0.601	0.358
مسيّرة (drone)	0.826	0.722	0.755	0.396
الكل (all)	0.744	0.592	0.651	0.377

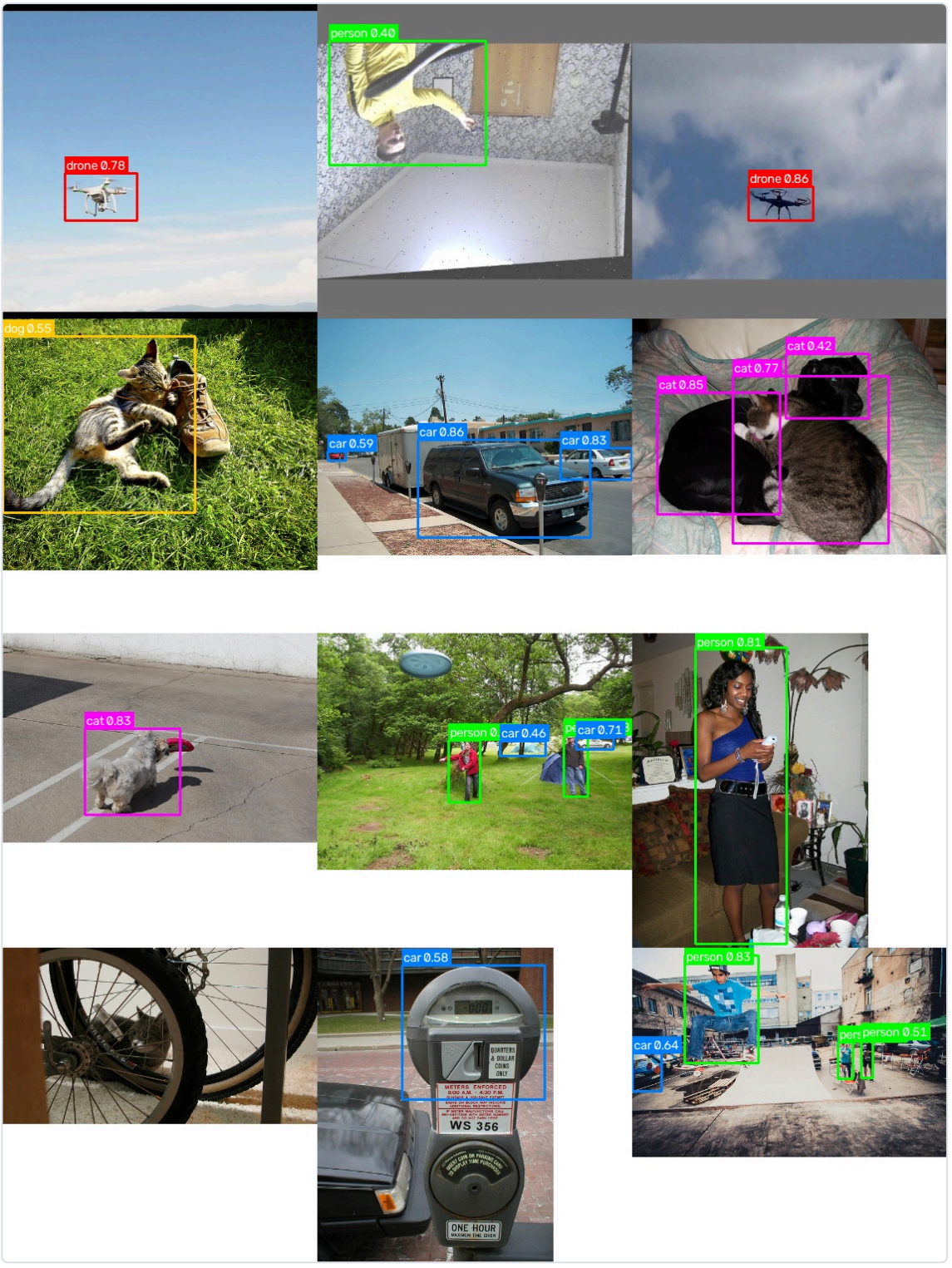
كيف نقرأ هذه الأرقام؟

- **المسيّرة (0.755) والقطة (0.768) الأفضل.** القطة غالباً جسم كبير وواضح. والمسيّرة في صورها المأخوذة عادة جسم معزول في السماء أو على العشب، فهي سهلة التمييز رغم قلة أمثلتها (585 مربعاً فقط في التدريب).
- **السيارة (0.503) الأضعف، وRecall عندها 0.462 فقط.** يعني أن النموذج يفوّت أكثر من نصف السيارات. غالباً هي سيارات صغيرة أو بعيدة أو مخفية جزئياً في خلفية الصور، وهي الآن **معلَّمة** (بعد تصحيح خطأ المرحلة الأولى) فتُحسب علينا. هذا تفسير محتمل لم نختبره.
- **الشخص (0.629):** أقل من نموذج الأشخاص الأول (0.865). لا يجوز مقارنة الرقمين مباشرة: اختبار النموذج الأول كان على أشخاص أكبر من 2% من الصورة فقط، واختبار النموذج الجديد يشمل كل شخص مهما صغر.
- **Validation (0.673) أعلى من Test (0.651).** كما في المرحلة الأولى: اخترنا أفضل جولة بناءً على Validation، فنتيجتها متفائلة قليلاً. رقم Test هو الذي يُعتمد.
- **الاختبار فيه أمثلة قليلة لبعض الفئات** (69 قطة و79 مسيّرة)، فرقم كل فئة منها غير دقيق جداً.



منحنيات تدريب النموذج متعدد الفئات. هذه المرة cls_loss ليس صفراً، لأن النموذج يجب أن يميّز بين 5 فئات.

3. (تابع) أمثلة كشف



أمثلة كشف للنموذج الموحّد على صور الاختبار (حد الثقة 0.4). كل فئة بلون مختلف.

أخطاء حقيقية في هذه الصور (مهم أن تفهمها)

- **خلط بين الفئات:** قطة صغيرة على العشب اكتُشفت كـ **dog**، وكلب أبيض صغير اكتُشف كـ **cat**. القطط والكلاب الصغيرة متشابهة في الشكل والفرو.
- **كشف خاطئ (False Positive):** عدّاد موقف سيارات اكتُشف كـ **car**، لأنه يظهر دائماً بجانب السيارات في صور التدريب.
- **جسم فائت (False Negative):** قطة تحت دراجة، مخفية جزئياً، لم تُكتشف. هذا بالضبط موضوع تجربة الإخفاء في الفصل 5.

4. الإخفاء الجزئي: الفكرة والكود

في الواقع، الأجسام غالباً مخفية جزئياً: شخص خلف سيارة، قطة تحت طاولة. المطلوب أن يكتشف النموذج **الجزء الظاهر**.

كيف نصنع بيانات مخفية؟

لا توجد مجموعة بيانات جاهزة بنفس فئاتنا تحدد نسبة الإخفاء. لذلك نصنعها: نغطي جزءاً من كل جسم بقطعة مقصوصة من **صورة أخرى**، فيبدو كأنه خلف شيء. والعلامة الجديدة = **الجزء الظاهر فقط**.

```
  (بعد إخفاء 40% من اليمين)        +-----------+  :قبل  +-------+#####+
        |   person  |                                    | ظاهر  |#####|
        +-----------+                                    +-------+#####+
                                                         العلامة الجديدة = الجزء الظاهر
فقط
```

scripts/make_occlusion.py

أ) مستطيل الإخفاء

```
def occluder_rect(box, side, ratio):
    _, x1, y1, x2, y2 = box
    bw, bh = x2 - x1, y2 - y1
    if side == "left":
        return x1, y1, x1 + bw * ratio, y2          # شريط من اليسار بعرض ratio من المربع
    if side == "right":
        return x2 - bw * ratio, y1, x2, y2
    if side == "top":
        return x1, y1, x2, y1 + bh * ratio
    return x1, y2 - bh * ratio, x2, y2              # من الأسفل
```

ب) حساب الجزء الظاهر

```
def visible_part(box, occ):
    c, x1, y1, x2, y2 = box
    ox1, oy1, ox2, oy2 = occ
    if ox2 <= x1 or ox1 >= x2 or oy2 <= y1 or oy1 >= y2:
        return box                                  # لا تقاطع ← المربع كما هو
    spans_h = oy1 <= y1 and oy2 >= y2               # الإخفاء يغطي كل ارتفاع الجسم؟
    spans_w = ox1 <= x1 and ox2 >= x2               # الإخفاء يغطي كل عرض الجسم؟
    if spans_h and spans_w:
        return None                                 # الجسم مخفي كله ← نحذف علامته
    if spans_h:
        if ox1 <= x1:
            x1 = max(x1, ox2)                       # مخفي من اليسار ← نحرّك الحافة اليسرى
        elif ox2 >= x2:
            x2 = min(x2, ox1)                       # مخفي من اليمين ← نحرّك الحافة اليمنى
    elif spans_w:
        ...                                         # نفس الشيء من الأعلى أو الأسفل
    if x2 - x1 < 4 or y2 - y1 < 4:
        return None                                 # أقل من 4 بكسل ظاهر ← نعتبره مخفياً
    return [c, x1, y1, x2, y2]
```

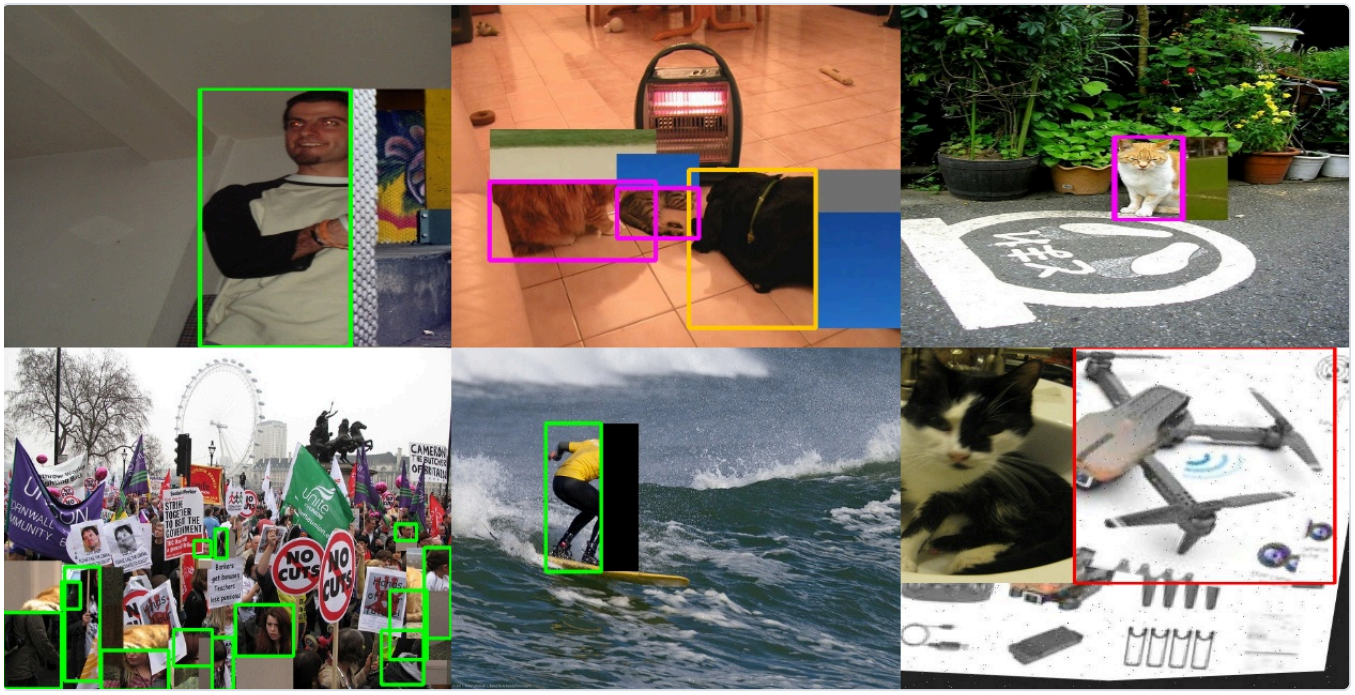
إذا غطّت القطعة جسماً آخر بجانب الجسم المستهدف، نصغّر مربعه بنفس الطريقة. إذا غطّت زاوية فقط، يبقى المربع كما هو، لأن حدود الجزء الظاهر لم تتغير.

ج) لصق القطعة

```
def paste_patch(img, rect, donor, rng):
    x1, y1, x2, y2 = (int(round(v)) for v in rect)
    ...
    sx, sy = rng.randint(0, dw - pw), rng.randint(0, dh - ph)   # مكان عشوائي في الصورة
المانحة
    img[y1:y2, x1:x2] = donor[sy:sy + ph, sx:sx + pw]          # نسخ القطعة فوق الجسم
```

القطعة من صورة حقيقية (وليست مستطيلاً أسود)، حتى يكون الإخفاء أقرب للواقع، ولا يتعلم النموذج حيلة سهلة مثل "المستطيل الأسود = إخفاء".

4. (تابع) أمثلة على البيانات المخفية



أمثلة بإخفاء 40%. المربعات = الجزء الظاهر فقط. في الصورة السفلى اليمنى: قطعة من صورة قطة تغطي يسار المسيّرة، فصار مربع المسيّرة على الجزء الأيمن الظاهر.

البيانات التي صنعناها

المجموعة	نسبة الإخفاء	الصور	مربعات ظاهرة	أجسام اختفت كلياً
اختبار 20%	20%	310	970	14
اختبار 40%	40%	310	931	53
اختبار 60%	60%	310	888	96
إضافة للتدريب	20–60% عشوائي	1212 (نصف صور التدريب)	3541	125

5. تجربة الإخفاء ونتائجها

تصميم التجربة

النموذج	كيف صُنع
الأساسي	النموذج الموحّد (30 جولة على صور عادية)
ضبط عادي (مجموعة ضابطة)	الأساسي + 10 جولات إضافية على الصور العادية فقط
ضبط مع الإخفاء	الأساسي + 10 جولات على الصور العادية + 1212 نسخة مخفية

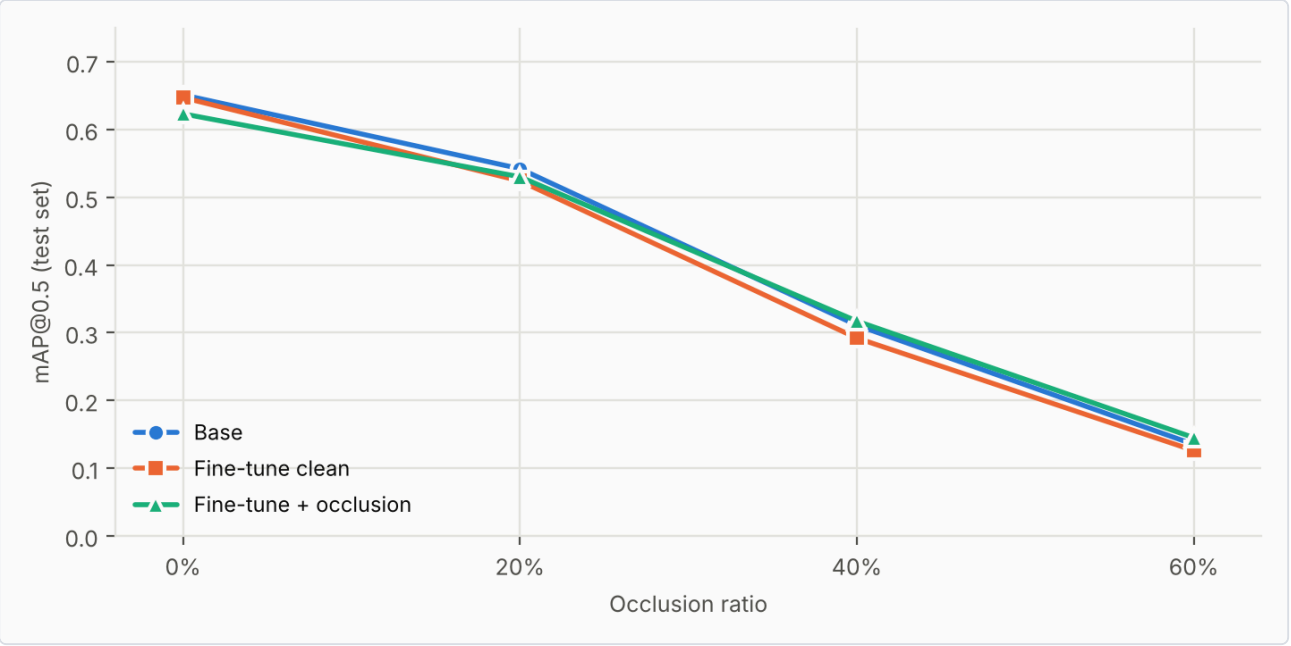
> **لماذا مجموعة ضابطة (Control)؟**
> لو قارنا "الأساسي" مع "ضبط مع الإخفاء" فقط، لن نعرف هل التحسن من الصور المخفية أم من الجولات العشر الإضافية. النموذجان المضبوطان لهما نفس البداية ونفس عدد الجولات ونفس seed، والفرق الوحيد هو بيانات التدريب.

```
python scripts/run_occlusion_experiment.py --base runs/train/multi/weights/best.pt
```

النتائج: mAP@0.5 على صور الاختبار

النموذج	إخفاء 0%	إخفاء 20%	إخفاء 40%	إخفاء 60%
الأساسي	**0.651**	**0.542**	0.311	0.135
ضبط عادي	0.647	0.524	0.292	0.126
ضبط مع الإخفاء	0.623	0.530	**0.317**	**0.145**

الرقم بالخط العريض = الأفضل في العمود. mAP@0.5:0.95 وRecall لكل حالة موجودة في `results/occlusion/occlusion.json`.



mAP@0.5 حسب نسبة الإخفاء للنماذج الثلاثة.

النتيجة 1: الإخفاء يضرّ كثيراً (واضحة وقوية)

النموذج الأساسي يهبط من **0.651** (بدون إخفاء) إلى **0.542** عند 20%، ثم **0.311** عند 40%، ثم **0.135** عند 60%. أي أنه يفقد حوالي **79%** من دقته عند إخفاء 60% من كل جسم. السبب أن الجزء الظاهر يخلو من أهم الملامح (الوجه، شكل الجسم الكامل)، وأن المربع الظاهر يختلف عن المربعات التي رآها النموذج في التدريب.

النتيجة 2: التدريب على صور مخفية ساعد قليلاً، وهذا غير حاسم

بمقارنة "ضبط مع الإخفاء" بالمجموعة الضابطة "ضبط عادي" (نفس البداية، 10 جولات، نفس seed):

- إخفاء 40%: **0.317** مقابل 0.292 (+0.025)، وإخفاء 60%: **0.145** مقابل 0.126 (+0.019)، وإخفاء 20%: 0.530 مقابل 0.524 (+0.006).
- على الصور العادية: 0.623 مقابل 0.647 (**−0.024**). أي أن النموذج صار أقل دقة قليلاً على الصور غير المخفية.

الاتجاه معقول (أكثر مقاومة للإخفاء الشديد، وأقل دقة قليلاً على الصور العادية)، لكن الفروق نحو **0.02 فقط** على 310 صور، وأجرينا **تجربة واحدة بـ seed واحد**، فلا نعرف حجم الضجيج العشوائي. **لا نستطيع القول إنه تحسّن حقيقي.** كلمة "ساعد" هنا تعني: الأرقام تميل في هذا الاتجاه.

لماذا الأثر صغير؟ (احتمالات، لم نختبرها)

- 10 جولات فقط من الضبط، ونموذج صغير (1.76 مليون معامل).
- الإخفاء الصناعي (قطعة من صورة أخرى) لا يشبه الإخفاء الحقيقي بدقة: في الواقع يكون الحاجز جسماً متصلاً له ظل وحدود طبيعية.
- عند 60%، يبقى من الجسم قليل جداً، وقد يكون ما بقي لا يكفي لأي نموذج.

ما الذي يقوّي الاستنتاج؟ تدريب أطول، وعدة seeds مع حساب المتوسط والانحراف، ونموذج أكبر، وبيانات إخفاء حقيقية (مثل CrowdHuman للأشخاص).

5. (تابع) النتائج حسب الفئة، وأمثلة

mAP@0.5 لكل فئة عند إخفاء 40%

الفئة	الأساسي	ضبط عادي	ضبط مع الإخفاء
شخص	0.267	0.246	0.275
سيارة	0.227	0.188	0.225
قطة	0.500	0.494	0.508
كلب	0.328	0.318	0.307
مسيّرة	0.231	0.214	0.273



نفس الصور المخفية (40%)، 6 صور بفواصل منتظمة من مجموعة الاختبار (غير منتقاة). في الأعلى النموذج الأساسي، وفي الأسفل المضبوط مع الإخفاء. الكشوف متقاربة جداً، وهذا يتفق مع الأرقام. الفرق الظاهر الوحيد: النموذج المضبوط اكتشف مسيّرة مخفية جزئياً (الصورة الأخيرة) فاتت الأساسي، وغيّر قطة/كلب إلى كلب/قطة في الصورة الثالثة (يبدو أسوأ).

6. خوارزميات الكشف الأخرى: النظرية

كل خوارزميات الكشف تجيب عن نفس السؤال: **أين الأجسام، وما نوعها؟** لكنها تختلف في الطريقة. هناك أربع عائلات رئيسية:

أ) الكاشفات ذات المرحلتين (Two-stage): عائلة R-CNN

الخوارزمية	السنة	الفكرة
R-CNN	2014	خوارزمية خارجية تقترح ~2000 منطقة، ثم شبكة CNN على كل منطقة وحدها. دقيقة لكن بطيئة جداً (ثوانٍ لكل صورة).
Fast R-CNN	2015	الشبكة تمر على الصورة مرة واحدة، ثم RoI Pooling يقتطع خصائص كل منطقة. أسرع بكثير.
Faster R-CNN	2015	شبكة اقتراح المناطق RPN جزء من الشبكة نفسها (المرحلة 1)، ثم رأس يصنّف ويضبط كل منطقة (المرحلة 2).

```
Faster R-CNN:
  الصورة ← Backbone (ResNet / MobileNet) ← خصائص
                          ↓
              المرحلة 1: RPN ← ~1000 "منطقة مرشحة" هنا جسم؟
                          ↓
              RoI Align ← قطع خصائص كل منطقة
                          ↓
              المرحلة 2: رأس ← الفئة + ضبط المربع
```

الميزة: دقة عالية، خاصة مع الأجسام الصغيرة. **العيب:** أبطأ، لأن هناك مرحلتين وآلاف المناطق.

ب) الكاشفات ذات المرحلة الواحدة (One-stage)

الخوارزمية	السنة	الفكرة
YOLO	2016+	شبكة واحدة، تمرير واحد، شبكة خلايا على 3 مقاييس (شرحناها في الملف الأول).
SSD	2016	مربعات مرجعية على خرائط خصائص بأحجام مختلفة. نسخة SSDlite مع MobileNet مصممة للهواتف.
RetinaNet	2017	اكتشفت سبب ضعف الكاشفات ذات المرحلة الواحدة: عدم التوازن بين آلاف مربعات الخلفية وقليل من الأجسام. الحل: **Focal Loss**.

Focal Loss باختصار

الخسارة العادية: $CE = -\log(p)$. الخسارة البؤرية: $FL = -(1-p)^{\gamma}\log(p)$.

عندما يكون النموذج واثقاً وصحيحاً (p قريب من 1، مثل آلاف مربعات الخلفية السهلة)، يصبح $(1-p)^{\gamma}$ صغيراً جداً، فتكاد الخسارة تختفي. هكذا يركّز التدريب على الأمثلة **الصعبة**.

ج) الكاشفات بدون مربعات مرجعية (Anchor-free): FCOS

بدل المربعات المرجعية، كل نقطة داخل الجسم تتنبأ مباشرة بأربع مسافات: إلى الحافة اليسرى والعليا واليمنى والسفلى (l, t, r, b)، مع قيمة "centerness" تقلل وزن النقاط البعيدة عن المركز. **الميزة:** لا حاجة لضبط أحجام المربعات المرجعية لكل مجموعة بيانات. YOLOv8 و YOLO11 تستعمل نفس الفكرة.

د) الكاشفات بالمحوّلات (Transformers): DETR

```
DETR:
  الصورة ← CNN ← خصائص ← Transformer Encoder (انتباه بين كل أجزاء الصورة)
                                          ↓
       100 "استعلام" (object queries) ← Transformer Decoder ← 100 تنبؤ (فئة + مربع)
                                          ↓
       كل جسم حقيقي ↔ تنبؤ واحد بالضبط: المطابقة الهنغارية (Hungarian matching)
```

الجديد: لا مربعات مرجعية، و**لا NMS:** كل جسم له تنبؤ واحد بفضل المطابقة الهنغارية أثناء التدريب. الانتباه (Attention) يرى الصورة كلها، فيفهم السياق. **العيب:** بطيء في التدريب (500 جولة على COCO) وفي الاستدلال، وضعيف نسبياً مع الأجسام الصغيرة.

ملخص العائلات

العائلة	أمثلة	السرعة	الدقة	Anchors	NMS
مرحلتان	Faster R-CNN	بطيء	عالية	نعم	نعم
مرحلة واحدة	YOLOv5، SSD، RetinaNet	سريع جداً	جيدة إلى عالية	نعم	نعم
بدون مربعات مرجعية	FCOS، YOLOv8	سريع	عالية	لا	نعم
محوّلات	DETR	بطيء	عالية	لا	**لا**

7. مقارنة الخوارزميات: النتائج

`scripts/compare_detectors.py`

طريقة المقارنة

- **نفس الصور:** 320 صورة من **COCO val2017** فيها أشخاص وسيارات وقطط وكلاب (1135 مربعاً).
- **لماذا val2017؟** كل النماذج الجاهزة تدرّبت على COCO train2017. لو اختبرناها على صور train2017 (مثل صور اختبارنا) لكانت قد رأت الصور من قبل، والمقارنة غير عادلة.
- **نفس طريقة الحساب:** mAP بطريقة COCO من مكتبة torchmetrics، لكل النماذج.
- **4 فئات فقط:** person, car, cat, dog. المسيّرة ليست في COCO، فالنماذج الجاهزة لا تعرفها.
- **السرعة:** على CPU بـ 4 أنوية، صورة واحدة في كل مرة.

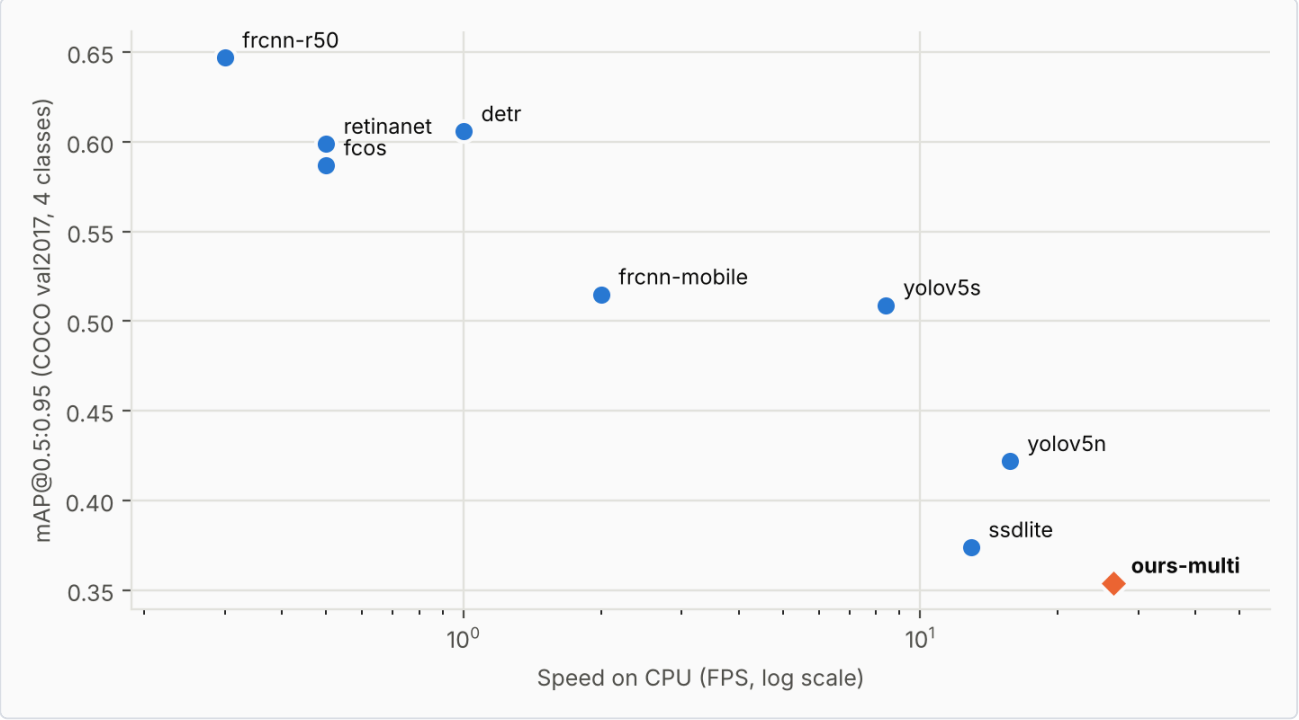
```
class TorchvisionDetector:
    def __init__(self, name):
        builders = {
            "ssdlite":      (d.ssdlite320_mobilenet_v3_large, ...COCO_V1),
            "retinanet":    (d.retinanet_resnet50_fpn_v2, ...COCO_V1),
            "fcos":         (d.fcos_resnet50_fpn, ...COCO_V1),
            "frcnn-mobile": (d.fasterrcnn_mobilenet_v3_large_fpn, ...COCO_V1),
            "frcnn-r50":    (d.fasterrcnn_resnet50_fpn_v2, ...COCO_V1),
        }
        build, weights = builders[name]
        self.model = build(weights=weights, ...).eval()     # أوزان مدرّبة مسبقاً على COCO

    def __call__(self, bgr):
        x = torch.from_numpy(rgb).permute(2, 0, 1).float() / 255   # صورة ← tensor و بين 0 و
1
        out = self.model([x])[0]                                     # boxes, scores,
labels
        return keep_ours(out["boxes"], out["scores"], out["labels"], COCO91_TO_OURS)
```

كل كاشف مغلّف بنفس الواجهة: صورة تدخل، و(boxes, scores, labels) بأرقام فئاتنا تخرج. هكذا تُقيَّم كل الخوارزميات بنفس الكود تماماً.

7. (تابع) جدول المقارنة

النموذج	العائلة	mAP@0.5	mAP@0.5:0.95	AP صغير	ms / صورة	FPS
Faster R-CNN ResNet50-FPN v2	مرحلتان	**0.883**	0.647	0.469	3151	0.3
DETR ResNet-50	Transformer	0.809	0.606	0.278	1043	1.0
RetinaNet ResNet50-FPN v2	مرحلة واحدة + Focal Loss	0.833	0.599	0.350	1850	0.5
FCOS ResNet50-FPN	مرحلة واحدة، بدون Anchors	0.821	0.587	0.337	1872	0.5
Faster R-CNN MobileNetV3-FPN	مرحلتان	0.774	0.515	0.221	496	2.0
YOLOv5s (COCO، 640)	مرحلة واحدة	0.764	0.509	0.354	118	8.4
YOLOv5n (COCO، 640)	مرحلة واحدة	0.672	0.422	0.071	64	15.7
SSDlite320 MobileNetV3	مرحلة واحدة	0.580	0.374	0.002	77	12.9
نموذجنا (YOLOv5n، 5 فئات، 416)	مرحلة واحدة (نموذجنا)	0.612	0.354	0.101	38	**26.5**



الدقة مقابل السرعة. أعلى اليمين = أفضل (دقيق وسريع). المحور الأفقي لوغاريتمي.

ماذا تقول النتائج؟

1. **الدقة مقابل السرعة: لا شيء مجاني.** الأدق (mAP@0.5:0.95 = 0.647) Faster R-CNN ResNet50 لكنه يحتاج **3.1 ثانية** للصورة على CPU (0.3 FPS). نموذجنا الأسرع **(FPS 26.5)** لكنه الأقل دقة في المقياس الأصعب (0.354).
2. **أفضل دقة لكل ثانية بين النماذج السريعة: YOLOv5s.** دقته (0.509) تقارب Faster R-CNN MobileNet (0.515)، لكنه أسرع **4 مرات** تقريباً (118 مقابل 496 ملّي ثانية).
3. **RetinaNet و FCOS و DETR متقاربة في الدقة** (0.59 إلى 0.61) وكلها بطيئة (0.5 إلى 1 FPS). أي أنه لم يظهر في تجربتنا أفضلية واضحة لعدم وجود Anchors (FCOS) على RetinaNet.
4. **الأجسام الصغيرة (AP صغير):** أضعف نموذج SSDlite (0.002، صورته المدخلة 320 فقط)، ثم YOLOv5n (0.071) مقابل (0.354) YOLOv5s: الأجسام الصغيرة تحتاج دقة صورة وسعة أكبر. (0.278) DETR أضعف من RetinaNet (0.350) و FCOS (0.337) فيها، وهذا يتفق مع ما هو معروف عن DETR.
5. **نموذجنا أقل من YOLOv5n الجاهز** (0.354 مقابل 0.422) رغم أن البنية **واحدة**. الفرق من التدريب والبيانات لا من البنية: YOLOv5n الجاهز تدرّب على 118 ألف صورة و80 فئة، وصورة بحجم 640. نحن دربنا على 2480 صورة فقط، 30 جولة، بحجم 416.

تحفّظات يجب ذكرها للأستاذ

- **320 صورة فقط وتشغيل واحد:** فروق صغيرة (مثل 0.509 و 0.515) ليست مؤكدة.
- **كل نموذج بحجم صورته الافتراضي** (YOLOv5 بـ 640، ونموذجنا بـ 416، و SSDlite بـ 320، والبقية بـ 800 تقريباً). لم نوحّد حجم الصورة، فجزء من الفرق في السرعة والدقة سببه حجم الصورة لا الخوارزمية.
- **السرعة:** على CPU بـ 4 أنوية، صورة واحدة في كل مرة، بدون GPU. على GPU ستتغير الأرقام كثيراً (خاصة للنماذج الثقيلة).
- **المسيّرة غير مقيَّمة** لأن النماذج الجاهزة لا تعرفها. الفئات الأربع فقط.

8. المشاكل التي واجهتنا

المشكلة	السبب	الحل
لا توجد فئة drone في COCO	COCO فيها 80 فئة ليس منها المسيّرات	مجموعة بيانات خارجية + تغيير رقم الفئة إلى 4
أشخاص بلا علامة في صور المسيّرات	المصدر يضع علامة على المسيّرات فقط	Pseudo-labeling بـ YOLOv5s (188 علامة)
أشخاص صغار بلا علامة (من المرحلة الأولى)	كنا نحذف علامات الأجسام الصغيرة ونُبقي الصور	كل الأجسام في الصورة المختارة تحصل على علامة
توقف التدريب عند الجولة 12	أُعيد تشغيل الجهاز	`train.py --resume last.pt`
`data.yaml 'train:' field missing`	val.py يطلب حقل train حتى عند التقييم فقط	إضافة `train: images` لملفات بيانات الاختبار المخفية
مقارنة غير عادلة	النماذج الجاهزة رأت صور COCO train2017	المقارنة على COCO val2017
النموذج لا يُحمَّل على Python 3.12	كائنات pathlib من Python 3.13 داخل الملف	`export_weights.py` يحفظ النموذج فقط

9. التشغيل بكاميرا الهاتف

```
cd "C:\Users\chems\real time humain detection\real_time-_object_detection"
git pull

# النموذج الموحّد: person, car, cat, dog, drone
python scripts/detect_realtime.py --weights models/multi_yolov5n.pt --source 1

# النموذج الأول (أشخاص فقط) ما زال موجوداً
python scripts/detect_realtime.py --weights models/person_yolov5n.pt --source 1
```

الفئة	لون المربع
person	أخضر
car	أزرق
cat	بنفسجي
dog	برتقالي
drone	أحمر

جرّب الإخفاء بنفسك

قف أمام الهاتف، ثم غطِّ نصف جسمك بكرسي أو باب أو ورقة كبيرة، ولاحظ هل يبقى المربع، وهل يتقلص إلى الجزء الظاهر. جرّب النموذجين (الأساسي والمضبوط مع الإخفاء) واكتب الملاحظات في التقرير.

النموذج المضبوط على الإخفاء موجود أيضاً

الملف `models/multi_occ_yolov5n.pt`. جرّبه بنفس الأمر (غيّر اسم الملف فقط). تذكّر أن فروقه عن النموذج الأساسي صغيرة في اختباراتنا، فلا تتوقع فرقاً كبيراً بالعين.

10. أسئلة للمراجعة مع الإجابات

1. لماذا اخترنا صور الفئة الأندر أولاً عند جمع البيانات؟

لأن صور الأشخاص كثيرة جداً، وكثير من صور القطط والكلاب فيها أشخاص أيضاً. لو بدأنا بالأشخاص لأخذنا صوراً كان يمكن أن تخدم الفئات النادرة.

2. لماذا يجب أن يحصل كل جسم في الصورة على علامة؟

لأن أي جسم بلا علامة يُعتبر "خلفية" أثناء التدريب، فيتعلم النموذج ألّا يكتشفه، ويعاقَب إذا اكتشفه.

3. ما هو Pseudo-labeling، وما خطره؟

استعمال نموذج جاهز لوضع علامات على بيانات ناقصة. خطره أن أخطاء النموذج المساعد تدخل البيانات، لذلك قبلنا فقط الكشوف بثقة ≥ 0.5.

4. لماذا تكون العلامة في تجربة الإخفاء على الجزء الظاهر فقط؟

لأن هذا ما طلبه الأستاذ (كشف الجزء الظاهر)، ولأن النموذج لا يستطيع أن يرى ما خلف الجسم الذي يغطيه. مربع على الجزء المخفي سيكون تخميناً.

5. لماذا الإخفاء بقطعة من صورة حقيقية، وليس بمستطيل أسود؟

حتى يكون أقرب للواقع، ولا يتعلم النموذج حيلة سهلة مثل "حافة سوداء = إخفاء" لا تفيد في الواقع.

6. لماذا احتجنا نموذجاً ضابطاً (ضبط عادي) في تجربة الإخفاء؟

حتى نعرف أن التحسن جاء من الصور المخفية، وليس من الجولات العشر الإضافية فقط.

7. ما الفرق بين الكاشف ذي المرحلتين والكاشف ذي المرحلة الواحدة؟

ذو المرحلتين (Faster R-CNN) يقترح مناطق أولاً ثم يصنّفها، فهو أدق وأبطأ. ذو المرحلة الواحدة (YOLO، SSD) يتنبأ بكل شيء في تمرير واحد، فهو أسرع.

8. ما المشكلة التي يحلها Focal Loss؟

عدم التوازن بين آلاف مربعات الخلفية السهلة وقليل من الأجسام. يقلل وزن الأمثلة السهلة حتى يركّز التدريب على الصعبة.

9. لماذا لا يحتاج DETR إلى NMS؟

لأن المطابقة الهنغارية أثناء التدريب تربط كل جسم حقيقي بتنبؤ واحد فقط، فيتعلم النموذج ألّا يكرر التنبؤ لنفس الجسم.

10. لماذا قارنّا الخوارزميات على COCO val2017 وليس على صور اختبارنا؟

لأن كل النماذج الجاهزة تدرّبت على COCO train2017، ومنها جاءت صور اختبارنا. لو اختبرناها عليها لكانت قد رأت الصور من قبل.

11. لماذا mAP لفئة person في النموذج الموحّد أقل من نموذج الأشخاص الأول؟

لأن النموذج الموحّد مطالب بكل الأشخاص في الصورة، حتى الصغار والبعيدين جداً، وهم صعبون. النموذج الأول كان يُختبر على أشخاص أكبر من 2% من الصورة فقط. الاختباران مختلفان، فلا يُقارنان مباشرة.